

```
struct VS_OUTPUT
```

```
{
```

```
    float4 Pos : SV_POSITION;
```

```
    float2 Tex : TEXCOORD0;
```

```
};
```

```
VS_OUTPUT VS(float4 Pos : POSITION, float2 Tex : TEXCOORD)
```

```
{
```

```
    VS_OUTPUT output = (VS_OUTPUT)0;
```

```
    output.Pos = Pos;
```

```
    output.Tex = Tex;
```

```
    return output;
```

```
}
```

```
// === НОВЫЙ КОД: ПРИНИМАЕМ ПАРАМЕТРЫ КАДРА ИЗ C++ ===
```

```
// Имя cbData может быть любым, главное — привязка к регистру b0 (слот 0)
```

```
cbuffer cbData : register(b0)
```

```
{
```

```
    float width;    // Ширина картинки (из C++ придет 2.0)
```

```
    float height;   // Высота картинки (из C++ придет 2.0)
```

```
    float d_width;  // Шаг пикселя по горизонтали (из C++ придет 0.5)
```

```
    float d_height; // Шаг пикселя по вертикали (из C++ придет 0.5)
```

```
};
```

```
// Константа весов для перевода RGB в монохромную яркость (Luma) из вашего 3-го шейдера
```

```
static const float3 WEIGHT = float3(0.299f, 0.587f, 0.114f);
```

```
// Пиксельный шейдер, подготовленный под 3-й проход (анализ блоков 4x4)
```

```
float4 PS(VS_OUTPUT input) : SV_Target
```

```
{
```

```
    // Тестовая проверка: давайте прямо в шейдере проверим, дошли ли до него
```

```
    // параметры d_width и d_height из C++ кода.
```

```
// Если данные дошли успешно — мы умножим их на 2 и окрасим экран в чистый серый цвет
половинной яркости!
```

```
float4 test_color = float4(d_width * 2.0f, d_height * 2.0f, d_width * 2.0f, 1.0f);
```

```
return test_color;
```

```
}
```